

CS F407 Artificial Intelligence – Week 2: Agents Laboratory

Advay Matapurkar | ID: 2023A7PS0459G

Constructing a goal-based agent (warehouse navigation) with an LLM

LLM used: Claude (Sonnet 5.5) via Claude Code. Files: `warehouse_agent.py` and `run_output.txt`.

Task 1 – Understanding the problem

1. Environment. A 7×21 grid (66 free cells including S and G). It is fully observable (the whole map is known), static (shelves never move), deterministic (a move always succeeds if the target is free), discrete, and single-agent.

2. Goal. Move the vehicle from S at (1,1) to the dispatch area G at (1,19) along a collision-free path; ideally the shortest one.

3. Actions. Up, Down, Left, Right, each moving one cell. An action is legal only if the target cell is inside the grid and is not '#'.

4. Information to maintain. The current position; the goal position; the map (to test legality of moves); the set of already-reached cells (to avoid loops); and, for each reached cell, the predecessor and action used to reach it (to reconstruct the path). The search frontier holds cells still to be expanded.

5. Why goal-based, not simple reflex. A reflex agent maps the current percept directly to an action through condition–action rules (e.g. 'if wall on right, go down'). That fails here: the map has dead ends and detours (the route must go around the shelf at column 6), so the correct move at a cell depends on where the goal is and on consequences several steps ahead. The agent has an explicit goal and *searches* over sequences of actions, judging them by whether they reach that goal.

Think about it – twice as large. Plain BFS would still be *correct* (complete and optimal for unit step costs) but its work grows with the number of cells: doubling each dimension gives about 4× the cells. We measured this: 59 expanded states for the original map versus 237 for the 2×2-scaled map (about 4×). For a much larger warehouse, memory for the frontier and visited set becomes the bottleneck, so a heuristic search such as A* (Manhattan distance) would be preferable. Further difficulties: partial observability, moving obstacles or other vehicles (forcing re-planning), different move costs (then use Dijkstra/A*), and multi-vehicle coordination.

Task 2 – Designing the agent

I produced this design myself, before giving the LLM any prompt; the Task 3 prompt was written from it. The design fixes the environment, state, goal, actions and the job of the decision-making component, but not the search algorithm: that was left to the LLM, which chose breadth-first search (Task 3).

Component	Design
Environment	Grid map: '#' obstacle, '.' free, S start, G goal
Current state	Vehicle position (row, col); initially S
Goal	Position == G (goal test)
Actions	Up, Down, Left, Right (one cell, only onto non-'#' cells inside the grid)
Decision-making	A search over positions from the current state to the goal that returns an action sequence, or 'no path'. Algorithm not fixed in the design; the LLM chose breadth-first search with a parent map.

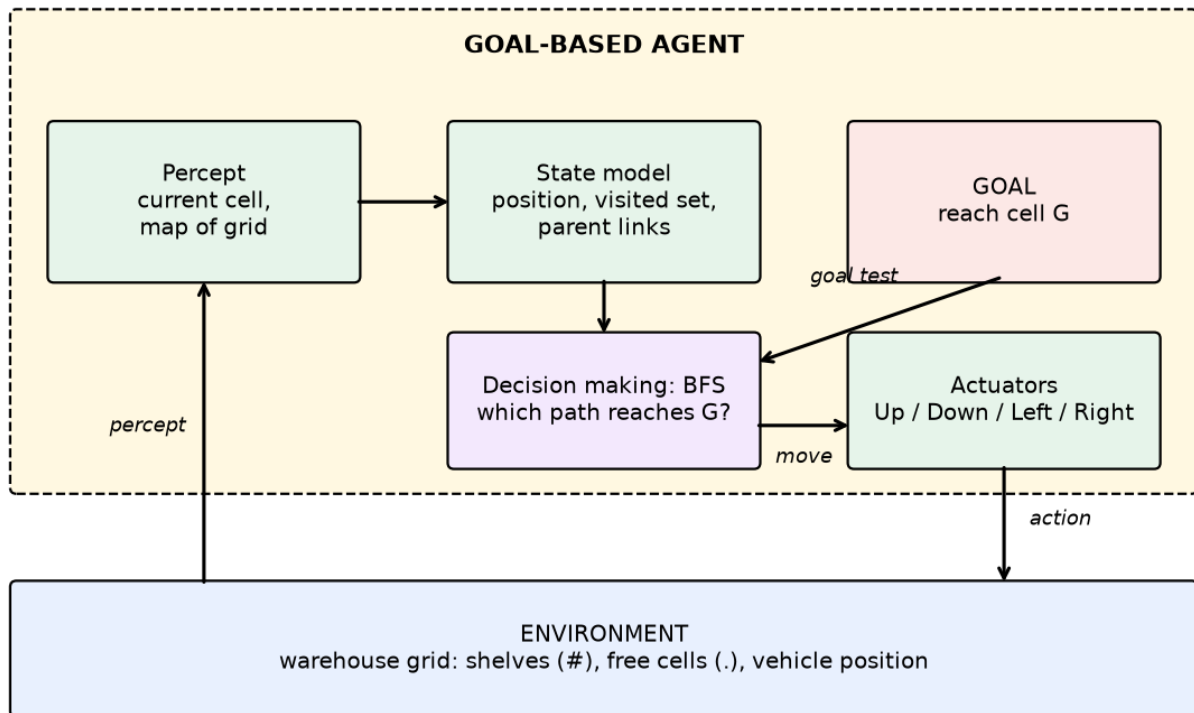


Figure 1. Block diagram of the goal-based agent. Percepts update the state model; the goal and the state feed the decision component, which selects actions executed in the environment. The diagram shows the final agent, so the decision box is labelled with the BFS that the LLM chose.

Task 3 – Prompt engineering

Prompt used (a specification rather than 'write a program'; it extends the suggested prompt with extra requirements):

Write a well-documented Python program implementing a goal-based agent for the warehouse navigation problem (map given, S=start, G=goal, #=obstacle, .=free; moves Up/Down/Left/Right, one cell each). The program should:

- represent the warehouse as a 2-D grid and validate the input map;
- determine a collision-free path from S to G, avoiding all obstacles;
- print the path (cells and actions) or a clear message if none exists;
- explain which search algorithm is used and why it is appropriate;
- include tests: the given map, an unreachable goal, a map twice as large, and malformed maps; and independently verify the found path.

Result of running the program (full log in run_output.txt). Path length 20 moves, 59 states expanded, verified to use legal moves only ('*' marks the path):

```
#####
#S**.*#####G#
#.*#####.#
#...##.....#
#.*#####.###.#.###.#
#.....#.....#
#####
```

Unreachable goal: 'No path exists'. 2× map: 40 moves. Malformed maps are rejected with a clear error.

Reflection questions

- **1. Working program on first attempt?** The agent itself (validation, BFS, path reconstruction) was correct and produced the optimal path on the first run. However, the *first full run did not complete*: the assistant's test code that builds the 'goal walled off' map by string slicing produced a non-rectangular map, so the agent's own validator raised an error and the script crashed. The assistant's second attempt, a scripted search-and-replace patch, also failed (the target text did not match). The third edit, building the

map from a list of characters, fixed it. So: core logic yes, whole program no – it took three tries. This shows the value of the validator and of running the code rather than trusting it.

- **2. How to improve the prompt?** State precisely how test maps are to be constructed (e.g. 'edit a list of lists, not string slices'), require rectangular-map validation with a clear message, ask for the tests to be run before the answer is returned, and specify the tie-breaking order of actions so the output is reproducible. Stating the optimality requirement ('shortest path') makes the algorithm choice explicit instead of implied.
- **3. Algorithm chosen.** Breadth-First Search (uninformed graph search with a visited set).
- **4. Why?** The problem has unit step costs and a small, fully known, static grid, so BFS is complete and guaranteed to return a shortest path, runs in $O(V+E)$ (here $O(\text{cells})$), and is simple to get right. DFS would not guarantee shortest paths; Dijkstra adds nothing without varying costs; A* is unnecessary overhead at this size, though it would be the natural upgrade for larger maps.

Critical evaluation of LLM-assisted development

- **Strengths:** fast, idiomatic and well-documented code; sensible algorithm choice; quickly generates tests and edge cases.
- **Limitations:** bugs can appear in the glue and test code rather than in the core algorithm; the first output is not guaranteed to run; the LLM does not know the intent unless the prompt states it. Every output must be executed and independently checked (here by the separate `verify()` function and the no-path and scaled-map tests).
- **Note:** I used Claude Code, which both wrote and ran the program, so the account of attempts above is taken from that session and not reconstructed afterwards.